
RLHF ALIGNMENT FOR GPT-OSS AND GEMMA3

Siddharth Menon

a1962276

Master of Artificial Intelligence and Machine Learning

Adelaide University

siddharth.menon@student.adelaide.edu.au

March 13, 2026

ABSTRACT

In this report, we examine and analyze how different models with different parameter sizes respond to different kinds of RLHF techniques. Reinforcement learning from Human Feedback is becoming more and more popular and has improved from the original *InstructGPT paper*[5] with the newly founded strategies like DPO[1] and GRPO[3]. We will select 2 use custom cases/datasets for this assignment. We will choose 2 models and DPO as our strategy to align these models with RLHF. In this way, we can analyze different combinations of datasets, models, and finetuning strategies. The 2 models we will be using in the experiments will be **GPT-OSS-20B** and **Gemma3-1B**. Both models are text-only but it is interesting to see how the parameter difference will come into play during finetuning. Libraries like *Unsloth* help very GPU efficient finetuning as they simplify complex processes like *gradient check-pointing*, *CPU offloading* and many other processes which will be discussed in this report.

1 LITERATURE

DPO[1](Direct Preference Alignment) is the main driver of this analysis. DPO suggests that there is no need for an explicit external *Reward Model* like in PPO[2] and we can use the base model itself as an implicit reward model. We also would not require to assign scores to these chosen and rejection pairs as the only thing we need to decide is the "*beta*" value. Most other methods will require us to train a separate reward model or would require us to score all the entries in the dataset. Choosing DPO as a strategy greatly simplifies the task. We will not require a *score* system while using DPO. DPO is commonly used in many RLHF[5] alignment tasks where we have access only to "chosen" and "rejected" pairs of data. For both the tasks, we will have to create these pairs since the datasets are custom. We have successfully aligned both outputs to a satisfactory level. The plots of reward and accuracy later in the report show this alignment as well. *Unsloth* and *TRL* are heavy drivers of this projects success. *Unsloth* allows for really good memory optimization and *TRL* is a user-friendly RLHF framework which abstracts a lot of the complex logic from the user and allows for quick and easy prototyping and training. The loss function of DPO is given as follows.

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

2 METHODOLOGY

2.1 How to process data into clean JSONs?

The 2 datasets used are for very different alignment tasks. The first dataset is a **Stackoverflow dataset** which contains information like "Question-Answer pairs", "Upvotes" for both questions and answers, "Tags" like *python*, *linux* etc and some meta-data which we will not be using. The second dataset is a **GenZ Slang dataset**. This is a simple dataset that has GenZ slang terms which can then be mapped to its simple english counterpart. I wrote python scripts to preprocess both datasets into the required DPO format. For using the DPO Trainer object from TRL we need the dataset in a simple

CSV format where we have 3 main columns. "Prompt" (the input), "Chosen" (the answer we want to align the model towards), and "Rejected" (the answer we want to steer away from). The StackOverflow dataset has a pipeline that is a little more complicated than the GenZ slang dataset. Here we go through all questions and answers, first we choose the questions with the highest upvotes, then we choose the answers with the highest upvotes for these selected questions. This ensures good quality training data as it is directly sampled using "Human Preference" as the criteria.

2.2 Reason(s) for selection of Models

The 2 models used for this assignment are **GPT-OSS-20B** and **Gemma3-1B-it**. These models are ideal for the task as Gemma is a small model with 1B parameters and GPT-OSS is 20x the parameters. We can use this to compare the behaviors(both training and evaluation) of 2 models that are of drastically different sizes. For this assignment we have chosen *text-only* LLMs to keep the training pipeline simple. Unsloth allows us to efficiently offload gradients and certain layers onto the CPU, this does increase the training and evaluation time but this allows us to perform all experiments on a consumer grade GPU like **NVIDIA GeForce RTX 3090** with **24GiB** of VRAM. Unsloth has a *FastLanguageModel* object which is inherited directly from huggingface transformers, which allows seamless integration with the models we are trying to use for the alignment task. This *FastLanguageModel* object has a function called *get_peft_model* which allows us to perform parameter efficient finetuning on the desired layers. The *get_peft_model* extracts and loads the LoRA modules for the required layers that we can specify as a list. The layers we will be training for both GPT-OSS and Gemma3 will be `["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`. The ranks we have tried for this experiment are 16 and 32 as these were the only ones possible for both models given the GPU constraint of 24GiB. The hierarchy of abstraction used for this project is as follows. The main overarching abstraction is Unsloth as we need to wrap the model with it. The DPO Object belongs to the TRL class. We pass the Unsloth wrapped model and the formatted data to the TRL class for training/alignment.

2.3 Training Pipeline

The main heavy lifting in the training pipeline is being done by TRL(Transformer Reinforcement Learning), a library from huggingface. Below is an example diagram of the entire Pipeline that we have used for the same.

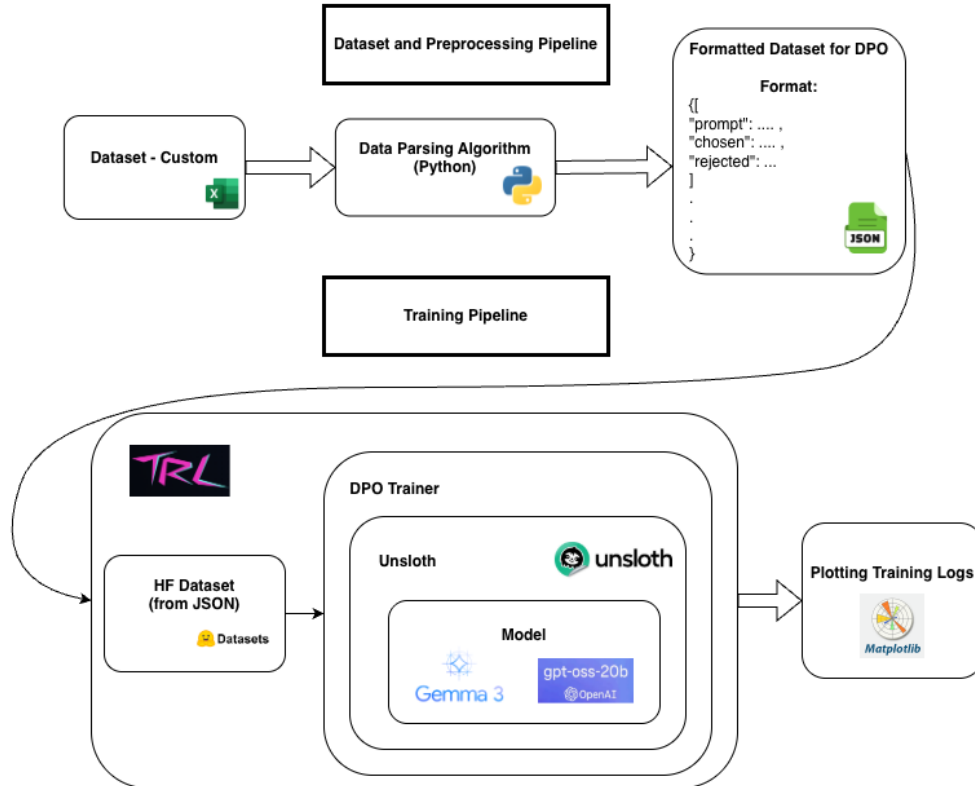


Figure 1: Diagram of the entire RLHF DPO Pipeline

Unsloth allows us to use quantized versions of the model. For this task, we used *4-bit* quantized versions of both models. This allows us to fit both models easily onto a 24GiB GPU and also since we use the 4-bit versions for both models, the comparison can be attributed purely to the number of parameters.

2.4 How to select parameters for model(s) across task(s)?

The most crucial parameter change with both datasets is the "*beta*" parameter. It determines how strictly the model adheres to its original behavior versus aligning with new preference data. We follow [this](#)[7] guide provided by openai for DPO finetuning. In the case of the stack overflow dataset we need the model to not forget too much of what it was trained on. It just needs to understand how Stack overflow answers are usually to the point and concise. We don't want the model to loose its reasoning/problem solving abilities because of this alignment training. This is why the "*beta*" is set to **0.5** for the Stack Overflow task. When it comes to the GenZ slang dataset. We need the model to not produce tokens like it used to, as the language expected has completely changed. So we use a small "*beta*" of 0.1 for this task. We try to keep most of the other parameters consistent across both models so that the change is performance/ behavior can be purely attributed to the model size.

Table 1: Parameters for Stack Overflow Dataset

	GPT-OSS-20B	Gemma3-1B
rank	16	32
grad acc steps	8	16
lr	5e-6	5e-6
beta	0.5	0.5
label smoothening	0.1	0.1
optimizer	adamw_8bit	adamw_8bit

Table 2: Parameters for GenZ slang Dataset

	GPT-OSS-20B	Gemma3-1B
rank	16	16
grad acc steps	16	16
lr	5e-6	5e-6
beta	0.1	0.1
label smoothening	0.1	0.1
optimizer	adamw_8bit	adamw_8bit

2.5 Suitable Evaluation Metric Selection

Below are the plots for the *training logs* for both the models across both the tasks. This has been taken from the [this](#) blog[6]. The effect of "*beta*" is very clearly visible in the Reward comparison graph of both the models across both the tasks. For the GenZ slang task, we can clearly see how the Chosen and Rejected probabilities diverge much better because of the lower beta value of 0.1. However, the stack overflow task is much more complex and requires the model to really learn the "essence" of what a good stack overflow answer is. This can be clearly seen in a qualitative analysis as well(discussed in the next section). The loss goes down very smoothly in the GenZ alignment task, since this is considerably simpler than in the stack overflow task. The Stack Overflow task has high variance and we can see that clearly in how much the loss values fluctuate. The metrics we keep track of here are, "**Training Loss**", "**Reward margin**", "**Reward Comparison**", and "**Reward Accuracy**". All metrics are plotted

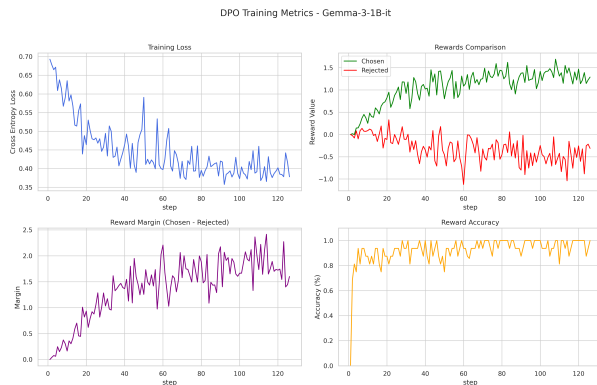


Figure 2: Gemma3-1B GenZ DPO Align Logs

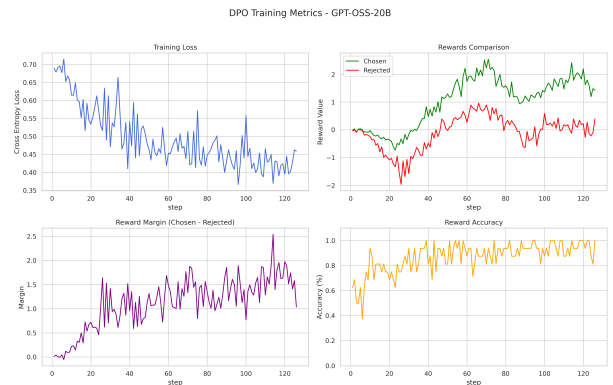


Figure 3: GPT-OSS-20B GenZ DPO Align Logs

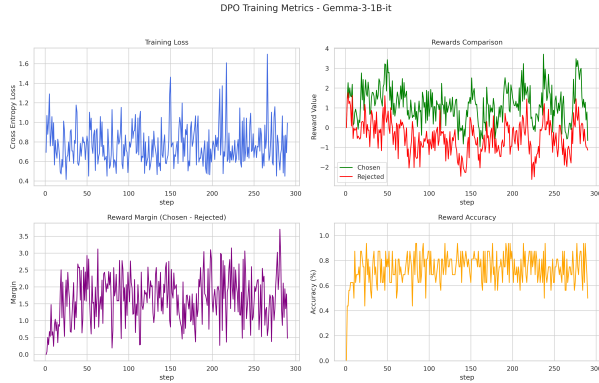


Figure 4: Gemma3-1B Stack Overflow DPO Align (Plot looks thicker because I logged more points)

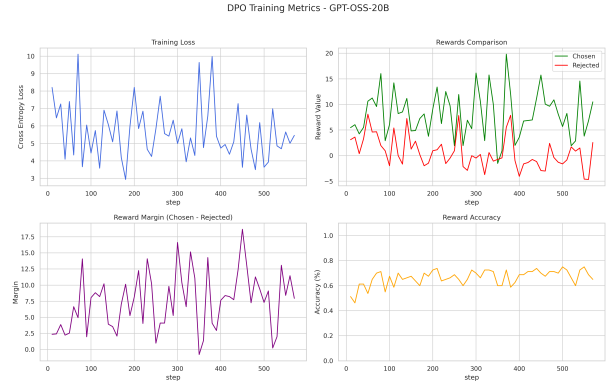


Figure 5: GPT-OSS-20B Stack Overflow DPO Align Logs

3 Qualitative Evaluation

In this section, we look at the **Vanilla Model output** vs **DPO-Aligned Model output**. We will use the same prompt and test it out on both the Models in both configurations. vanilla refers to the "off-the-shelf" model from huggingface. The head to head comparison serves as a great qualitative metric for understanding the alignment.

In case of the **first task**(GenZ slang alignment), we used '<CUSTOM> slang' in the prompt to make sure the LLMs do not rely on its existing knowledge of what "GenZ slang" might mean. We can clearly see the effect of this alignment as the output definitely uses more *trendy* words like "rad", "fam", "straight-up" etc. For the **second task**(Stack Overflow alignment), we can see how the vanilla LLM has the tendency to over explain the solution and offer a lot of *meta* data along with the answer. We can see how the aligned output is more concise and gets to the point much quicker which aligns with how the top answers are on stack overflow.

3.1 GenZ Slang Alignment

Prompt: How do you say 'This is a very impressive achievement' in <CUSTOM> slang?

GPT-OSS-20B

Gemma3-1B

Vanilla Output

Yo, that's a total win-straight up impressively slick, bro.

Vanilla Output

Hey, that's a seriously cool accomplishment, seriously impressive!

DPO-Aligned

In <CUSTOM> slang you'd say, "That's a straight-up mad epic win, fam!"

DPO-Aligned

Hey, that's totally rad, a massive win!

3.2 Stack OverFlow Alignment

We will ask the same question for both GPT-OSS and Gemma3. It is a simple pandas column filtering question.

Prompt: You are an expert software engineer. Provide concise, accurate solutions. Jump directly to the answer. No meta-analysis.

"Tags:python, pandas

Title:How to filter a dataframe by date range?

Body:I have a dataframe with a 'Date' column. How do I get rows between 2023-01-01 and 2023-12-31?"

Model: GPT-OSS-20B**VANILLA GPT-OSS**

```
import pandas as pd

# Assume 'df' already exists and has a column named 'Date'
# Make sure it's of dtype datetime64
df["Date"] = pd.to_datetime(df["Date"])

start = pd.Timestamp("2023-01-01")
end = pd.Timestamp("2023-12-31")

# Boolean mask - keep rows whose date falls in the interval [start, end]
mask = (df["Date"] >= start) & (df["Date"] <= end)

rows_between_dates = df.loc[mask]      # this is your filtered DataFrame
```

If you prefer slicing on an indexed date column, first set it as the index:

```
df.set_index("Date", inplace=True)
subset = df.loc[start:end]             # automatically includes both bounds
```

Either approach will return all rows where **Date** lies between **1 Jan 2023** and **31 Dec 2023**, inclusive.

DPO-ALIGNED GPT-OSS

```
import pandas as pd

df = pd.read_csv('your_file.csv')      # your DataFrame
df['Date'] = pd.to_datetime(df['Date']) # ensure proper dtype

start_date = "2023-01-01"
end_date = "2023-12-31"

mask = (df["Date"] >= start_date) & (df["Date"] <= end_date)
filtered_df = df.loc[mask]
```

'pd.to_datetime()' converts the **Date** column to 'datetime64'. The Boolean mask keeps all rows whose timestamp falls within the inclusive interval from **1 Jan 2023** to **31 Dec 2023**, and 'loc[]' returns those rows.

Model: Gemma3-1B**VANILLA GEMMA 3**

```
import pandas as pd
# Assuming your DataFrame is named 'df'
df = pd.DataFrame({'Date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05'],
                   'Value': [10, 20, 30, 40, 50]})
# Filter for dates from 2023-01-01 to 2023-12-31
filtered_df = df[pd.to_datetime(df['Date'])]
# Print the filtered DataFrame
print(filtered_df)
```

Explanation:

- **import pandas as pd**: Imports the Pandas library, which provides data manipulation tools.
- **df = pd.DataFrame(...)**: Creates a sample DataFrame (replace this with your actual DataFrame). It includes a 'Date' column.
- **df['Date'] = pd.to_datetime(df['Date'])**: Converts the 'Date' column into datetime objects. This is crucial because Pandas needs to understand the date format correctly when filtering.
- **df[pd.to_datetime(df['Date'])]**: This is the core of the filtering operation. It uses boolean indexing to select rows where the 'Date' column falls within the specified range ('2023-01-01' to '2023-12-31'). Pandas automatically handles the comparison of datetime objects.
- **print(filtered_df)**: Prints the resulting DataFrame containing only the desired rows.

DPO-ALIGNED GEMMA 3

```
import pandas as pd
# Assuming your DataFrame is named 'df'
df = pd.DataFrame({'Date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05'],
                  'Value': [10, 20, 30, 40, 50]})
filtered_df = df[pd.to_datetime(df['Date'])] # Convert 'Date' column to datetime objects first
print(filtered_df)
```

Explanation:

- `**df['Date'] = pd.to_datetime(df['Date'])**`: This line converts the 'Date' column into datetime objects. Pandas automatically handles this if the string format is consistent. If your dates aren't consistently formatted, you might need more complex parsing logic using `'pd.to_datetime()'` or other methods for proper conversion.
- `**df[pd.to_datetime(df['Date'])]**`: This uses boolean indexing (also known as "loc" or "bracket notation") to select rows where the 'Date' column matches the specified date range. `'pd.to_datetime(df['Date'])'` converts the 'Date' column to datetime objects so that we can compare them. The square brackets `'[]'` create a copy of the DataFrame containing only the desired rows.

This approach efficiently filters the DataFrame based on the date range provided. The result will be a new DataFrame containing only the rows matching the date criteria.

Link to adapter weights for both models and tasks *Gemma3-1B-GenZ_task*, *GPT-OSS-20B-GenZ_task*, *Gemma3-1B-StackOverflow_task*, *GPT-OSS-20B-StackOverflow_task* is in this [Link](#) This needs to be downloaded and that path can be used in the inference code.

4 Limitations & Future Scope

1. **Mixed Precision Training:** Both models exploded/vanished in gradients when we tried to train them on *bfloat16* datatype. This might be due to the low precision stability of *bf16* as a datatype for training. This could also be because of how unsloth or TRL handles such data types. The observation was made by monitoring `'grad_norm'` value during training, which often went to *inf* or *nan* when using *bf16*.
2. **Dataset Size/ Variance control:** The GenZ dataset had around 1000 entries which helps alignment as the variance is very less for such dataset sizes. But the Stack Overflow dataset, even after taking out only the 99th percentile, had around 5000 rows, which causes very high variance at each step. This causes gradients to update with very high values. We can control this by adjusting the learning rate. But the learning rate is already very low($5e-6$).
3. **DPO strategy:** DPO [1] as an RLHF strategy is notorious for overfitting very quickly and "reward hacking" to maximize rewards if trained for too many epochs. We had to limit all experiments to 1 epoch or 2 epochs at best because of the same. In the GenZ dataset, if we went above 4 epochs, the model starts to output emojis only as it sees high reward for emojis. This loses the purpose of alignment and is an unfavorable outcome.
4. **Unsloth and TRL:** As good and easy as these frameworks are to use, they are "black boxes" and offer very little customization and visibility of the internal logic. Sometimes, the right optimization is hidden under these layers of abstractions.
5. **GRPO and GSPO:** Group Relative Policy Optimization[3] and Group Sequence Policy Optimization [4] are more SOTA algorithms that are being used currently in models like DeepSeek and Qwen and have shown extremely good results in performance of both these models. Although GRPO is supported by TRL(experimental), we are leaving this open for future scope.

5 Conclusions

We have successfully aligned both **GPT-OSS-20B** and **Gemma3-1B** for 2 different tasks. **Gemma3-1B** was clearly the **better model** for such downstream finetuning tasks. It showed much smoother convergence and was overall much more "well aligned" to the task given, both qualitatively and according to the metrics. We **cannot** make a conclusion however as maybe GPT-OSS required more training due to its large size. Choosing a value for "**beta**" seems to be one of the most crucial aspects for a DPO based RLHF alignment on a given task. Gradient accumulation helps overcome variance to some level as we can see, it helps smoothen out the convergence. **Unsloth** plays a vital role in helping fit everything into a 24GB VRAM machine, without which this project would not be possible. **Data pre-processing** is relatively easy as it is mostly just a format change and can be done with basic python parsing scripts. **Training times** vary based on the model and batch size, but for my configuration it took around 6h for Gemma3 and 9h for GPT-OSS finetuning.

References

- [1] Rafailov, R., Sharma, A., Mitchell, E., Manning, C.D., Ermon, S. and Finn, C., 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36, pp.53728-53741.
- [2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [3] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y.K., Wu, Y. and Guo, D., 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- [4] Zheng, C., Liu, S., Li, M., Chen, X.H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A. and Zhou, J., 2025. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*.
- [5] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A. and Schulman, J., 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35, pp.27730-27744.
- [6] Lambert, et al., "Illustrating Reinforcement Learning from Human Feedback (RLHF)", Hugging Face Blog, 2022.
- [7] BLOG by OpenAI: https://developers.openai.com/cookbook/examples/fine_tuning_direct_preference_optimization_guide